

169. Majority Element Easy

Acceptance: 66.0% | [LeetCode](#)

Given an array `nums` of size `n`, return *the majority element*. The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

```
**Input:** nums = [3,2,3]
**Output:** 3
```

Example 2:

```
**Input:** nums = [2,2,1,1,1,2,2]
**Output:** 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-109 <= nums[i] <= 109`
- The input is generated such that a majority element will exist in the array.

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

Claude Opus Solution

Majority Element — Optimal Solution

1. Intuition

The follow-up asks for **$O(n)$ time and $O(1)$ space**. The hash table approach is $O(n)$ time but uses $O(n)$ space. The key insight is the **Boyer-Moore Voting Algorithm**.

Core idea: The majority element appears more than $\lfloor n/2 \rfloor$ times — more than *all other elements combined*.

Imagine pairing up each majority element with a non-majority element so they "cancel each other out." Since the majority element appears more than half the time, after all possible cancellations, the majority element is the only one left standing.

We simulate this with a **candidate** and a **count**: - When we see our candidate, we add a "vote" (increment count). - When we see a different element, we remove a "vote" (decrement count) — a cancellation. - When count hits 0, the current candidate has been fully canceled, so we pick a new candidate.

Because the majority has a strict majority of votes, it can never be fully canceled out — it survives.

2. Approach (Pseudocode)

```
candidate = None
count = 0

for num in nums:
    if count == 0:           # no current candidate, pick this one
        candidate = num

    if num == candidate:    # vote for candidate
        count += 1
    else:                   # vote against -> cancel one
        count -= 1

return candidate           # guaranteed to be the majority
```

3. Python Solution

```
from typing import List

class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        # `candidate` holds the current potential majority element.
        # `count` tracks the "net votes" for the candidate.
        candidate = None
        count = 0
```

```

for num in nums:
    # If count is 0, all previous votes canceled out.
    # Adopt the current number as the new candidate.
    if count == 0:
        candidate = num

    # If this number matches the candidate, it's a supporting vote (+1).
    # Otherwise, it's an opposing vote that cancels one out (-1).
    if num == candidate:
        count += 1
    else:
        count -= 1

# Since a majority element is guaranteed to exist (appears > n/2 times),
# it cannot be fully canceled out, so `candidate` is the answer.
return candidate

```

Why it's guaranteed correct here

The majority element appears $> n/2$ times. Even if every other element votes against it, there are fewer than $n/2$ opposing votes — not enough to cancel out the majority's $> n/2$ supporting votes. So the candidate at the end must be the majority.

⚠️ Note: If a majority element were *not* guaranteed, you'd need a second pass to verify the candidate actually appears $> n/2$ times. The problem guarantees existence, so we skip that.

4. Complexity Analysis

Metric	Value	Explanation
Time	$O(n)$	Single pass through the array; each element processed in $O(1)$.
Space	$O(1)$	Only two variables (<code>candidate</code> , <code>count</code>) regardless of input size.

This satisfies the follow-up's linear-time, constant-space requirement — strictly better than the hash table's $O(n)$ space.

5. Alternative Approaches

Approach	Time	Space	Notes
Brute Force (reference)	$O(n^2)$	$O(1)$	Count occurrences of each element. Too slow for large inputs.
Hash Table (reference)	$O(n)$	$O(n)$	Count frequencies in a dict. Simple but uses extra space.
Sorting	$O(n \log n)$	$O(1)^*$	After sorting, the element at index <code>n//2</code> is always the majority. Elegant one-liner: <code>return sorted(nums)[len(nums)//2]</code> . (* $O(1)$ only if sorting in place.)
Boyer-Moore Voting ⭐	$O(n)$	$O(1)$	The optimal solution above.
Bit Manipulation	$O(n \cdot 32)$	$O(1)$	For each of the 32 bits, count set bits across all numbers; if more than $n/2$ have a bit set, that bit belongs to the answer. Works because the majority dominates each bit position.

Recommended: Boyer-Moore Voting — it's the canonical optimal answer and a great pattern to know for stream-processing / counting-dominance problems.

Sorting one-liner (clean alternative)

```

class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        # After sorting, the majority element (>n/2 occurrences)
        # must occupy the middle index no matter where it lies.
        nums.sort()
        return nums[len(nums) // 2]

```

Codex Solution

1. Intuition

The key insight is:

If an element appears more than $n / 2$ times, it cannot be fully canceled out by all the other elements.

This leads to the **Boyer-Moore Voting Algorithm**.

Think of the majority element as having more "votes" than every other element combined.

We keep:

- a `candidate` for the majority element
- a `count` representing how strong the current candidate is

When we see the same number as the candidate, we increase `count`.

When we see a different number, we decrease `count`, meaning one occurrence of the candidate is "canceled" by one occurrence of a different number.

If `count` becomes `0`, we choose a new candidate.

Because the majority element appears more than half the time, it will survive all cancellations.

2. Approach

Boyer-Moore Voting Algorithm

```
Initialize candidate = None
Initialize count = 0
```

```
For each num in nums:
    If count == 0:
        candidate = num
        count = 1
    Else if num == candidate:
        count += 1
    Else:
        count -= 1
```

```
Return candidate
```

Since the problem guarantees that a majority element always exists, we do not need a second pass to verify the candidate.

3. Python Solution

```
from typing import List

class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        # Boyer-Moore Voting Algorithm
        #
        # candidate stores the current possible majority element.
        # count tracks the "vote balance" for the candidate.
        candidate = None
        count = 0

        for num in nums:
            # If count is 0, we have no active candidate.
            # Choose the current number as the new candidate.
            if count == 0:
                candidate = num
                count = 1

            # If the current number matches the candidate,
            # it supports the candidate, so increase count.
            elif num == candidate:
```

```
        count += 1

        # Otherwise, this number cancels out one vote
        # from the current candidate.
    else:
        count -= 1

    # The problem guarantees that a majority element exists,
    # so the final candidate must be the majority element.
    return candidate
```

4. Complexity Analysis

Time Complexity: $O(n)$

We iterate through the array exactly once.

```
n = length of nums
```

So the time complexity is:

```
 $O(n)$ 
```

Space Complexity: $O(1)$

We only use two extra variables:

- candidate
- count

No hash map or extra array is used.

So the space complexity is:

```
 $O(1)$ 
```

5. Alternative Approaches

1. Brute Force

For every element, count how many times it appears in the array.

```
for num in nums:
    if nums.count(num) > len(nums) // 2:
        return num
```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

This is simple but inefficient.

2. Hash Map Counting

Count the frequency of every number using a dictionary.

```
from collections import defaultdict

count = defaultdict(int)

for num in nums:
    count[num] += 1
    if count[num] > len(nums) // 2:
        return num
```

Complexity

- Time: $O(n)$

- Space: $O(n)$

This is easy to understand but does not satisfy the $O(1)$ space follow-up.

3. Sorting

If the array is sorted, the majority element must be at index $n // 2$.

```
nums.sort()
return nums[len(nums) // 2]
```

Complexity

- Time: $O(n \log n)$
- Space: $O(1)$ or $O(n)$ depending on the sorting implementation

This works because an element appearing more than half the time must occupy the middle position after sorting.

Key Pattern

This problem is a classic example of:

Vote cancellation / Boyer-Moore Majority Vote

Use this when:

- You need to find an element appearing more than $n / 2$ times
- The majority element is guaranteed to exist
- You want $O(n)$ time and $O(1)$ space